CivicEye: A Java Swing Interface for Civic Issue Reporting and Role-Based Resolution Tracking

A CBP PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING

BY

NAME OF THE STUDENT 1 25071A0501
NAME OF THE STUDENT 2 25071A0502
NAME OF THE STUDENT 3 25071A0503
NAME OF THE STUDENT 4 25071A0504

UNDER THE GUIDANCE OF
Mr. I Ravindra Kumar
Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
V ALLURUPALLI NAGESWARA RAO
VIGNANA JYOTHI INSTITUTE OF ENGINEERING & TECHNOLOGY
Vignana Jyothi Nagar, Bachupally, Hyderabad 500118, Telangana, India

October 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the CBP Project titled “CivicEye: A Java Swing Interface for Civic Issue Reporting and Role-Based Resolution Tracking” is a record of bona fide work submitted by:

- NAME OF THE STUDENT 1 25071A0501
- NAME OF THE STUDENT 2 25071A0502
- NAME OF THE STUDENT 3 25071A0503
- NAME OF THE STUDENT 4 25071A0504

in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology (B.Tech.) in COMPUTER SCIENCE AND ENGINEERING of Jawaharlal Nehru Technological University Hyderabad (JNTUH), under VNRVJIET B.Tech. R25 Academic Regulations. The work has been carried out under our guidance and supervision at Vallurupalli Nageswara Rao Vignana Jyothi Institute of Engineering & Technology, Hyderabad.

The report is original, free from plagiarism, and complies with the Institute guidelines, including those related to academic ethics and AI-generated content.

Signature of Supervisor	**Signature of HOD**
Mr. I Ravindra Kumar	**Dr. V Baby**
Assistant Professor	**HOD**
Department of CSE	**Department of CSE**

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

We, the undersigned, declare that the Project titled "CivicEye: A Java Swing Interface for Civic Issue Reporting and Role-Based Resolution Tracking" has been carried out by us in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology (B.Tech.) in COMPUTER SCIENCE AND ENGINEERING of Jawaharlal Nehru Technological University Hyderabad (JNTUH), under VNRVJIET B.Tech. R25 Academic Regulations, at Vallurupalli Nageswara Rao Vignana Jyothi Institute of Engineering & Technology, Hyderabad.

We further declare that this work is original, has not been submitted previously for any degree or similar title, and complies with the Institute's plagiarism and academic ethics guidelines, including AI-generated content. The contributions of all team members are clearly indicated in the report.

PLACE: HYDERABAD

DATE: _____

Signature of the Student(s)

NAME OF THE STUDENT 1 25071A0501

NAME OF THE STUDENT 2 25071A0502

NAME OF THE STUDENT 3 25071A0503

NAME OF THE STUDENT 4 25071A0504

ABSTRACT

Citizens report civic problems such as broken footpaths, failed street lights, blocked drains, and uncollected garbage, but the interface through which those reports are made often decides whether the report is usable at all. A confusing form, an unresponsive window, or a status that is never visible turns a simple complaint into a frustrating experience and leaves the reporting office without a reliable record. CivicEye addresses this problem as a self-contained desktop application built with Java Swing, in which the citizen and the administrator each receive a purpose-built workspace and every action produces immediate, unambiguous visual feedback. The interface follows a consistent dark visual language, uses standard Swing components arranged with layout managers rather than absolute positioning, keeps long-running work off the event dispatch thread, and reflects the permitted issue lifecycle directly in the controls that are offered. Phase-II work delivered the login and registration windows, the citizen dashboard with a validated submission form and a personal issue register, the administrator dashboard with a status filter and a selection-driven update panel, custom table renderers that draw status badges, and an empty-state view for first-time users. Verification combined automated checks of the application logic with a structured walkthrough of every screen: eighty-four assertions passed with no failures, and each interface flow produced the expected visual state. The project shows that careful use of standard Swing components, disciplined event handling, and consistent feedback can produce an interface that is both pleasant to use and difficult to misuse.

Keywords: *Java Swing, Desktop user interface, Civic issue reporting, Event handling, Role-based interface, Usability*

TABLE OF CONTENTS

CERTIFICATE **2**

DECLARATION **3**

ABSTRACT **4**

CHAPTER 1 - INTRODUCTION **6**

CHAPTER 2 - LITERATURE REVIEW **11**

CHAPTER 3 - APPROACH AND DESIGN METHODOLOGY **16**

CHAPTER 4 - DEVELOPMENT AND IMPLEMENTATION **23**

CHAPTER 5 - RESULTS, ANALYSIS AND VALIDATION **34**

CHAPTER 6 - CONCLUSIONS AND FUTURE SCOPE **41**

REFERENCES **45**

APPENDIX A: CONTRIBUTION OF TEAM MEMBERS **47**

CHAPTER 1 - INTRODUCTION

1.1 Background of the Problem

Civic infrastructure is maintained by municipal bodies that must respond to a continuous stream of small but consequential complaints. A damaged footpath, an overflowing drain, a street light that has not worked for a week, or uncollected garbage in a residential lane are individually minor events, but in aggregate they determine whether a neighbourhood feels cared for or abandoned. Traditionally these complaints reach the municipality through a telephone call, a written letter, a visit in person, or a message passed to a ward representative. Each of these channels is conversational and ephemeral, and none of them gives the citizen a durable artefact such as a reference number or a visible status that can be checked later.

The growth of digital government has changed expectations on the citizen side. Layne and Lee describe electronic government as a staged progression from simple information publishing, through transaction processing, to integration across departments. Most municipalities in India have completed the first stage with informational websites, and many have moved to the second stage with grievance portals and mobile applications. Yet the quality of the interface varies enormously. A portal may accept a complaint and then show nothing except a generic thank-you page, leaving the citizen unable to tell whether the report was registered, ignored, or duplicated. The interface, in other words, is not a cosmetic layer around the real system; it is the part through which trust is either earned or lost.

A second issue concerns the office rather than the public. The person who actually triages complaints works at a desk, often on a shared computer, and needs a workspace that makes the next action obvious. If the administrative interface is an afterthought, the clerk must open several screens, remember identifiers, and retype information, which is where errors and delays originate. Good administrative interfaces reduce the number of decisions a user must hold in memory at once by showing the selected record, its current state, and the permitted next actions together in one place.

CivicEye was conceived against this background as a deliberately small desktop application written in Java Swing. It accepts reports only from authenticated users, presents a citizen with a form for submitting a complaint and a register of everything they have already reported, and presents an administrator with the complete register and a controlled way to advance a report through its lifecycle. The two roles see different workspaces because they have different jobs, and the visual design makes that difference obvious from the moment the window opens.

1.2 Need and Relevance

The need for a clear interface is first a need for traceability in the user's own hands. When a complaint is submitted, the citizen should immediately see an identifier, a timestamp, and a status, and should be able to reopen the application later and find the same record unchanged. CivicEye provides this through a register table that lists every report submitted by the logged-in user, newest first, with a coloured status badge that can be read at a glance. The table is not an optional extra; it is the evidence that the report exists.

The second need is error prevention. Forms that accept impossible input waste the citizen's time and create work for the office. CivicEye constrains the category to a fixed list, enforces a minimum and maximum description length, shows a live character counter, disables actions while a submission is in progress, and reports the specific problem next to the form rather than in a modal dialog that must be dismissed. Nielsen's heuristic of preventing errors rather than reporting them is applied literally: the interface makes the invalid action difficult before it makes the error message necessary.

The third need is responsiveness. A window that freezes while work is happening feels broken even if the work will eventually succeed. Every action in CivicEye that could take perceptible time runs in a background worker, the initiating control is disabled and relabelled while the work is in flight, and the result is delivered back to the interface only when it is known. This pattern is applied to login, registration, submission, refresh, and status update, so the interface never stops painting.

The fourth need is affordability and continuity. A desktop client runs on an ordinary computer, does not require a public domain, and keeps working when the network is unavailable. For a ward office or a campus administration that already owns a computer, the marginal cost of deployment is close to zero. The trade-off is that the application cannot be reached from a citizen's phone, which is why mobility is treated as future scope rather than as a launch requirement.

1.3 Problem Statement

The problem addressed by this project is the absence of a simple, role-aware, and visually unambiguous desktop interface for recording civic complaints and following their resolution in settings that cannot support a full web platform. Complaints arrive through unstructured channels, are recorded inconsistently or not at all, and are closed without the citizen ever seeing a status change. The person at the desk has no single screen that shows what is open, what was just reported, and what may be done next.

Stated precisely, the interface must provide a citizen workspace in which a report can be composed, validated, submitted, and then found again in a personal register; an

administrator workspace in which the whole set of reports can be reviewed, filtered, and advanced through a permitted sequence of states; and a consistent feedback language in which success, failure, and in-progress conditions are always visible. The interface must also prevent the most common mistakes by construction, rather than relying on the user to read an error message after the fact.

The constraints that shape the solution are equally important. The application must run on a single ordinary workstation with Java installed, must be implementable by a four-member student team within one semester, and must not depend on paid services or external interface frameworks that obscure the underlying mechanics. These constraints rule out a distributed web application and favour a compact Swing client with a clear separation between the windows and the logic they invoke.

1.4 Objectives

The objectives below translate the problem statement into observable interface behaviour. Each is stated so that it can be demonstrated in the running application or verified by an automated check, and each is revisited in Chapter 5 where the evidence for its fulfilment is presented.

- To build a desktop interface that starts reliably on an ordinary workstation and presents a coherent visual identity across every window.
- To provide role-based workspaces in which a citizen and an administrator see different screens, different actions, and different data.
- To implement a submission form that constrains the category, enforces length limits, shows a live character count, and reports validation problems inline.
- To present a citizen with a personal register of reports showing category, location, description, status, and submission time in a scannable table.
- To give the administrator a complete register with a status filter and a detail panel that shows the selected report and the permitted next actions.
- To keep the interface responsive by running every potentially slow action in a background worker and updating the interface only on the event dispatch thread.
- To produce immediate, unambiguous feedback for success, failure, and in-progress conditions without blocking modal interruptions.
- To verify the interface through a structured walkthrough of every screen, supported by automated checks of the application logic behind it.

These objectives are deliberately narrow. Photographic evidence, geographic coordinates, notification, and a public web client were considered and deferred, because each would

substantially increase the interface surface without strengthening the central claim of the project, which is that a carefully built Swing interface can make civic reporting clear and difficult to misuse. The deferred items are documented as future scope in Chapter 6.

1.5 Scope of Work

The scope of Phase-II covers the complete graphical application: the startup sequence, the login window, the registration dialog, the citizen dashboard, the administrator dashboard, the supporting panels, the custom renderers, and the verification walkthrough. The delivered system is functional end to end on a single workstation, and every screen has been exercised with realistic data. The report documents the presentation layer and the application logic that the windows invoke; components outside those layers are outside the scope of this document.

The following capabilities are inside the scope of the delivered interface:

- A login window with inline validation, a busy state, and a clear failure message.
- A modal registration dialog that reports the specific reason a registration was rejected.
- A citizen dashboard combining a submission form and a personal register in a draggable split.
- A validated report form with a fixed category list, a location field, a description area with a live counter, and separate submit and clear actions.
- A register table with sorting, custom status badges, truncated descriptions with tooltips, and a friendly empty state.
- An administrator dashboard with a status filter, a selectable register, and a detail panel that offers only the permitted next states.
- Keyboard support for refresh with the F5 key and confirmation before logout.
- A structured verification of eighty-four automated assertions plus a screen-by-screen walkthrough.

The following capabilities are explicitly outside the scope of this phase: geographic maps, image or document attachments, email and SMS notification, a browser-based or mobile client, multi-tenant support for several municipalities, and analytics beyond simple counts. These exclusions keep the project honest about what has been demonstrated and define a clear boundary for future work.

1.6 Organization of the Report

Chapter 2 reviews the literature on electronic government, interface usability, and desktop application design, compares the principal delivery approaches, and identifies the gap that CivicEye addresses. Chapter 3 describes the design methodology, the screen flow, the component structure, and the justification for the chosen tools. Chapter 4 documents the implementation of each window, the interaction between components, and the practical difficulties encountered during development. Chapter 5 presents the verification methodology, the measured results, an interpretation of those results, and a validation of each objective. Chapter 6 states the conclusions, acknowledges the limitations of the work, and outlines future scope. References and the team contribution appendix follow the chapters.

CHAPTER 2 - LITERATURE REVIEW

2.1 Overview of Existing Work

Research on electronic government provides the conceptual frame for this project. Layne and Lee propose a four-stage maturity model in which public agencies move from catalogue information, to transaction processing, to vertical integration within a functional area, and finally to horizontal integration across functions. Civic issue reporting sits in the second and third stages: it is a transaction that creates a record, and its value depends on that record being usable by the department that performs the repair. The model explains why so many municipal portals stop at form submission. A form that accepts text and then shows nothing has completed a transaction on paper but has not given the citizen anything they can act on.

Moon examines the adoption of e-government among municipalities and distinguishes between rhetoric and substantive change. His survey found that many portals improved information delivery without altering internal processes, so the promised transparency did not materialise. The same distinction applies to interfaces: a modern-looking page that hides the actual state of a complaint is less useful than a plain window that shows it clearly. The lesson adopted in CivicEye is that the visible state of a report deserves as much design attention as the form used to create it.

A second body of work studies citizen interaction and satisfaction. Reddick, analysing citizen-initiated contact with government, finds that convenience, responsiveness, and trust strongly influence whether citizens use a digital channel again. Welch, Hinnant, and Moon show that satisfaction is associated with perceived responsiveness and with trust in government more broadly, which implies that a complaint that disappears without a reply does measurable damage beyond the single incident. Belanger and Carter document the digital divide in e-government use, noting that access, skill, and cost shape who benefits from online services. These findings support a design that keeps the client lightweight, keeps the number of steps small, and makes status visible without requiring the citizen to learn a complex interface.

Civic technology practice offers concrete reference systems. FixMyStreet, operated by mySociety, lets residents report local problems and routes them to the responsible council; its enduring contribution is the public map of reports and the expectation that a report receives a reference and an update. In the United States, the 311 non-emergency system consolidated dozens of municipal service requests into a single channel. In India, the Swachhata-MoHUA platform handles sanitation grievances at national scale, and several municipal corporations run their own complaint portals with photographs and location

capture. These systems demonstrate the value of a single visible register, but they also depend on web infrastructure and mobile clients that small offices cannot always replicate.

The human-computer interaction literature supplies the design vocabulary used in this project. Nielsen's ten heuristics, and the usability engineering practice built around them, emphasise visibility of system status, match between the system and the real world, user control, consistency, error prevention, recognition over recall, and helpful error messages. Shneiderman and colleagues develop these ideas into a design framework covering layout, navigation, and interaction styles, while Norman explains why visible affordances and immediate feedback reduce the mental effort of using a tool. CivicEye applies these principles directly: status badges, disabled controls during work, inline messages, and a consistent palette are not decoration but the mechanisms through which the interface explains itself.

On the technical side, the Java Swing literature describes a mature toolkit built on the model-view-controller idea. Oracle's tutorial and the standard references by Horstmann and Eck document layout managers, event dispatch, table renderers, and the single-thread rule that governs component updates. Gamma and colleagues supply the pattern vocabulary used to separate the windows from the logic they call, and Sommerville and Pressman provide the incremental process and verification discipline followed during development.

2.2 Comparison of Methods and Approaches

Civic reporting can be delivered through several distinct channels, and each channel carries different costs and capabilities. The comparison below evaluates the principal options against the requirements that matter for a small municipality: the quality of visual feedback, the separation of roles, the infrastructure required, the ability to operate without connectivity, and the effort needed to build and maintain the interface.

Table 2.1 Comparison of civic complaint channels and delivery approaches

Approach	Visual feedback	Role separation	Infrastructure	Offline use	Build effort
Paper register or telephone helpline	None	None	None	Yes	Very low
Municipal web grievance portal	Depends on page design	Usually a separate login area	Server, hosting, browser	No	High
Mobile application with backend	Rich, but small screen	Separate role flows	Server, app distribution	Partial	High
Desktop client with a local register (this work)	Immediate and consistent	Distinct workspaces per role	One PC with Java installed	Yes	Moderate

The table makes the trade-off explicit. Web portals and mobile applications reach more citizens and support richer media, but they require continuous hosting, a responsive layout for many screen sizes, and a maintenance budget that small bodies often lack. A desktop client sacrifices reach but keeps the interaction predictable, remains usable during connectivity failures, and allows a dense administrative view that would be cramped on a phone. The trade-off is acceptable when the primary user is the ward office clerk rather than the general public.

A second comparison concerns the toolkit. JavaFX offers richer graphics, declarative layout, and modern controls, but it requires an additional runtime module and a different deployment story. Web frameworks offer the widest reach but introduce a server, a browser, and a build pipeline. Swing is older, plainer, and sometimes verbose, yet it ships with every Java runtime, works identically on the major desktop platforms, and exposes its event model clearly enough to be taught. For a project whose claim is clarity of interaction rather than novelty of technology, the transparent option is preferable.

2.3 Critical Analysis of Literature

The literature converges on feedback as the decisive variable. Welch, Hinnant, and Moon find that satisfaction depends less on the sophistication of the channel than on whether the citizen perceives the agency as responsive, and Nam makes the same point from the perspective of citizen sourcing: participation is sustained only when contributions produce visible outcomes. Nielsen's first heuristic, visibility of system status, states the same principle at the level of a single screen. This convergence suggests that a modest interface with honest, immediate feedback can outperform a rich interface that leaves the user guessing.

The transparency literature adds a caution. Bertot, Jaeger, and Grimes argue that information technology can foster a culture of transparency, but only when the published information is timely, complete, and usable. An interface that silently discards a submission, or that shows a success message before the work has actually succeeded, produces the appearance of transparency while undermining it. Jaeger and Bertot further warn that unequal access to digital channels can entrench existing disadvantages, which is a reason to keep the client inexpensive and to avoid assuming constant connectivity.

The usability literature qualifies the enthusiasm for visual richness. Shneiderman and colleagues note that consistency, informative feedback, and error prevention matter more than decoration, and Norman argues that affordances must be visible rather than memorised. A dark theme with rounded buttons and gradients is pleasant, but it earns its place only if it also communicates state: which field has focus, which action is available, and which status a report currently holds. CivicEye therefore treats colour as a signal,

reserving green for success, red for failure, amber for administrative identity, and blue for the primary action.

A critical reading of accessibility guidance adds a further caution. WCAG and ISO 9241-210 both stress that interfaces must accommodate users with different abilities, preferences, and contexts. Desktop applications are frequently exempted from the accessibility scrutiny applied to websites, even though office staff may rely on keyboard navigation and screen readers. CivicEye provides keyboard refresh and clear focus indication, but it has not been audited against an accessibility standard, and its custom-painted components may not expose full semantics to assistive technology. This gap is acknowledged in Chapter 6 rather than glossed over.

A final observation concerns sustainability. The civic technology literature contains many pilot applications that were abandoned once the initial funding or student cohort moved on. Interfaces that survive tend to use standard components, avoid exotic dependencies, and degrade gracefully on older machines. CivicEye's reliance on the Java standard library and its single driver-free runtime are deliberate responses to this pattern, even though the project itself remains a demonstration rather than a deployed municipal service.

2.4 Research Gaps Identified

The review identifies several gaps that motivate the present work. The first is the gap between public submission and visible accountability. Most published case studies describe portals that accept complaints and publish aggregate statistics, but few describe an interface in which the citizen can see the current state of an individual report, and fewer still constrain the administrator to permitted transitions. The second is the gap between the polish of public-facing screens and the neglect of administrative screens, where the actual work is done. The third is the scarcity of reference implementations whose interface code is small enough to read and reuse as teaching material.

- Small municipalities and campus administrations lack an affordable, role-aware desktop interface for recording civic complaints.
- Administrative screens are often designed last, so the person doing the triage must hold identifiers and states in memory across several dialogs.
- Status values are frequently free text or unconstrained selections, which makes the interface unable to show a reliable lifecycle.
- Validation feedback is commonly delivered as a blocking dialog after submission rather than as guidance during entry.
- Civic technology reports rarely document interface verification, so claims of usability cannot be checked or reproduced.

CivicEye responds to these gaps with four concrete decisions: a separate workspace for each role, a register table that makes status visible at a glance, a submission form that prevents invalid input before it is sent, and an administrator panel that offers only the permitted next states. The contribution is not a novel interaction technique but a complete, inspectable arrangement of standard Swing components and well-established usability principles for a setting that the literature tends to overlook.

CHAPTER 3 - APPROACH AND DESIGN METHODOLOGY

3.1 Overview of Work and Outcomes

The project followed an incremental and iterative methodology. Each increment produced a runnable slice of the interface, and each slice was reviewed before the next was started. This ordering was chosen because the riskiest parts of a Swing application are not the individual controls but the interactions between them: how a window hands control to another window, how a table selection populates a detail panel, and how background work returns to the event dispatch thread without corrupting the display. Building those paths early, in a thin vertical slice, exposed design problems while they were still cheap to correct.

The work was organised into seven increments whose outcomes are summarised below. The phases correspond to the milestones recorded in the project repository history, which is why the numbering begins with foundations rather than with requirements.

Table 3.1 Increments, deliverables, and completion status

Phase	Focus	Principal deliverable	Status
1	Foundations	Project structure, shared constants, visual palette, startup sequence	Complete
2	Identity screens	Login window, registration dialog, credential utilities, focus styling	Complete
3	Interaction logic	Validation rules, authentication flow, issue service, service checks	Complete
4	Data presentation	Issue model, table loading, refresh mechanism, row mapping	Complete
5	Citizen experience	Citizen dashboard, submission form, personal register, empty state	Complete
6	Administration	Administrator dashboard, status filter, detail panel, update flow	Complete
7	Verification and polish	End-to-end walkthrough, reward-points removal, screenshots, documentation	Complete

The cumulative outcome is a working desktop application of approximately four and a half thousand lines of Java across twenty-four source files, of which the graphical layer accounts for well over half. The application compiles from source with a single javac invocation and runs with a single java invocation. Every screen has been exercised in a structured walkthrough, and the supporting logic checks are reported in Chapter 5.

3.2 Refinement of Problem and Objectives Based on Feedback

Three refinements emerged from review feedback during the semester. The first concerned a reward points feature that had been implemented in an early version to encourage participation. Reviewers observed that points could be gamed, that they had no meaning for the municipality, and that they cluttered the citizen dashboard with a second row of statistics. The feature was therefore removed completely rather than hidden behind a flag, and the dashboard was simplified to the two things a citizen actually needs: a form and a register.

The second refinement concerned the status workflow. The initial administrator panel allowed any of the three statuses to be chosen at any time, so a resolved issue could be quietly returned to pending. Feedback pointed out that this destroys the evidential value of the record. The panel was changed in two ways: the status selector now offers only the states that are legal from the current one, and the application logic rejects every other combination with a specific message. The visual effect is that the interface itself teaches the permitted sequence rather than relying on the user to remember it.

The third refinement was the addition of a status filter to the administrator register. Without it, an administrator facing a growing list had to scan every row to find open work. The filter is a simple combo box above the table, and it re-renders the same list without another round of loading. A smaller refinement followed from the first walkthrough: when the register is empty, the table is replaced by a friendly panel that explains how to add the first report, rather than showing an intimidating grid of empty rows.

3.3 Selection of Approach and Methodology

The design separates the windows from the logic they invoke. The presentation layer consists of four windows and three reusable panels built from standard Swing components; the application layer contains authentication, issue handling, and validation; the model layer contains the domain entities and the role hierarchy; and a small support layer holds shared constants, colour definitions, and the credential utility. Dependencies point in one direction: a panel may call a service, a service may use the model, and nothing in the application layer knows that Swing exists.

This arrangement was selected over two alternatives. A single-class prototype with all logic inside action listeners would have been faster to write but impossible to test without a display, and it would have produced a window that could not be reused or extended. A JavaFX application would have offered richer styling and declarative layout, but it would have introduced a different runtime, a different threading model, and a deployment problem for laboratory machines. The chosen middle ground is small enough to be read in

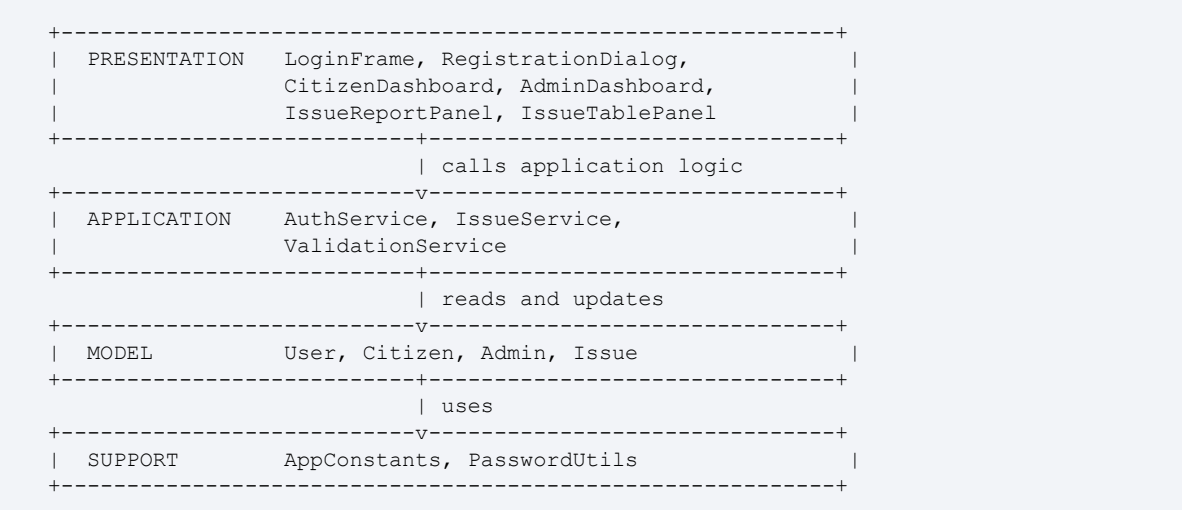
its entirety, explicit enough to be examined in a viva, and structured enough to be extended.

Within each increment, development followed a build-and-review discipline. Because the project does not use a build tool or a test framework, the logic checks are ordinary Java classes with a main method that exercise the layer beneath the interface and print a pass or fail line for every assertion. This choice keeps the barrier to running them at zero, at the cost of some duplication in the assertion helpers. The interface itself is verified through a documented walkthrough of each screen, described in Chapter 5.

Version control was used throughout with a linear history and descriptive commit messages that name the increment completed. The repository contains only source, scripts, documentation, and screenshots; compiled classes and local configuration are excluded by the ignore rules. This discipline matters because the project is meant to be reproducible by someone who downloads it without prior knowledge of the team's machines.

3.4 Technical Approach and Key Elements

The technical approach can be understood from three views: the layering of the application, the flow between screens, and the structure of the individual windows. Each view is presented below, and the implementation of each element is described in Chapter 4.



The package structure mirrors the layers, and the responsibilities of each package are summarised in Table 3.2. The separation is enforced by convention rather than by module boundaries, since the project compiles as a single source tree, but it is consistently respected: no window contains business rules, and no service imports a Swing class.

Table 3.2 Package responsibilities

Package	Responsibility	Representative classes
civiceye.main	Application entry point and event-thread bootstrap	Main
civiceye.ui	Windows, panels, renderers, and user interaction	LoginFrame, AdminDashboard, IssueTablePanel
civiceye.service	Business rules, authorisation, validation	AuthService, IssueService, ValidationService
civiceye.model	Domain entities and role polymorphism	User, Citizen, Admin, Issue
civiceye.util	Constants, palette, credential helper	AppConstants, PasswordUtils
civiceye.test	Standalone verification programs	PasswordUtilsTest, Phase3Test, EndToEndTest

The screen flow is linear at the start and then branches by role. The application opens a login window. A new user may open a modal registration dialog from that window and return to it on success. Authentication produces either a citizen or an administrator object, and the login window disposes itself and opens the corresponding dashboard. Each dashboard offers a logout action that closes the session window and returns to a fresh login window. There is no shared navigation bar between the two dashboards, because the two roles never need to reach each other's screens.

```
[Login] --create account--> [Registration dialog]
   |                                |
   |<--------- success -----------+
   |
   +-- citizen --> [Citizen dashboard: form | my register]
   |
   +-- admin ----> [Admin dashboard: filter | all reports | update]
                        |
                        +-- logout --> [Login]
```

Each window is composed from standard containers. The login and registration screens use a gradient background panel with a centred card built from a grid bag layout. The dashboards use a border layout with a header, a central content area, and a status bar. The citizen dashboard splits its content horizontally into a form and a register; the administrator dashboard splits vertically into a register and a detail panel. This uniformity means that a user who has seen one screen can predict where the header, the primary action, and the status line will be on the next.

The lifecycle of a report is expressed in the interface as well as in the logic. Pending reports are drawn with an amber badge, reports in progress with a blue badge, and resolved reports with a green badge. The administrator selector is populated from the current status, so the permitted next states are the only ones the user can choose, and a rejected transition produces a red message in the detail panel without changing the table. This is the visual counterpart of the rule that a resolved report is terminal.

```
Pending  --------->  In Progress  --------->  Resolved
   |                                            ^
   +--------------------------------------------+
          (direct resolution permitted)
Backward and no-op transitions are not offered by the interface.
```

3.5 Supporting Analysis and Design Considerations

Layout was treated as a design problem rather than as an afterthought. Every container uses a layout manager: border layout for the window frame, grid bag layout for forms, grid layout for paired buttons, and split panes for resizable regions. No component is positioned with absolute coordinates. This choice means that the interface survives resizing, different font sizes, and the minor metric differences between platforms, and it avoids the common defect in which a window looks correct only at the size at which it was designed.

Sizing was chosen deliberately. The login window is fixed at a compact size because it contains one card and should not be resized. The dashboards are resizable but declare minimum sizes so that the header buttons, the table columns, and the form fields cannot be squeezed into illegibility. The citizen form has a preferred width of about a third of the window, and the split pane lets the user adjust that ratio. The administrator table is given all surplus vertical space through a resize weight, and the detail panel keeps a fixed minimum height.

Concurrency and responsiveness were considered explicitly. Swing is single-threaded: all component mutations must occur on the event dispatch thread, while any slow operation blocks painting if it runs there. Every operation that could take perceptible time is therefore wrapped in a SwingWorker whose background method performs the work and whose completion method updates the interface on the event thread. Buttons are disabled and relabelled while work is in progress, which prevents duplicate submissions. This pattern appears in login, registration, issue submission, table refresh, and status update.

Feedback follows a consistent language. An in-progress state is a disabled control with a present-participle label, such as Submitting. A success is a green line containing the identifier of the affected report. A failure is a red line containing the specific reason, and the focus returns to the field that needs attention. A rejected status transition is reported in the detail panel without clearing the selection, so the user can immediately choose a different action. Nothing important is communicated through colour alone; every badge and message also carries text.

Accessibility was considered within the limits of the toolkit. The interface supports keyboard traversal through the standard focus system, provides a function-key shortcut for refresh, and draws a visible focus underline on every input. Contrast was checked by eye against the dark backgrounds rather than measured against a formal standard. The

custom-painted buttons and badges do not expose additional semantics to assistive technology, which is recorded as a limitation in Chapter 6 rather than presented as a completed accessibility claim.

3.6 Tools and Techniques Used

The toolchain was chosen for transparency and reproducibility. The project uses only the Java standard library and the Java Development Kit; no external interface library, styling framework, or build tool is required. The table below records the specific tools used during development and verification.

Table 3.3 Tools and techniques

Tool or technique	Version or detail	Purpose in the project
Java Development Kit	JDK 26 toolchain (javac 26.0.1)	Compilation and execution of all sources
Java Swing	Part of Java SE	Windows, panels, tables, custom painting
AWT and Java2D	Part of Java SE	Layout, fonts, colours, gradient and shape rendering
SwingWorker	Part of Java SE	Background work with completion on the event thread
Git	Distributed version control	Incremental history and milestone commits
javac and java	Command-line invocation	Build and run without an external build tool
Screen capture	Operating-system screenshot tool	Figures used in this report

Two techniques deserve separate mention because they shape the codebase. The first is the use of a shared constants class for every label, category, limit, and dimension, so that a change such as adding a category propagates to the form without touching the layout code. The second is the deliberate reuse of a small set of style helpers, such as the input field factory and the primary button factory, which keeps the visual language consistent across the login, registration, and dashboard screens. Both techniques reduce the opportunity for the interface to drift out of alignment as features are added.

3.7 Justification of the Selected Approach

The chosen approach is justified by the constraints established in Chapter 1. A Java Swing client satisfies the offline and cost requirements without compromise and runs on any machine with a standard Java installation. A layered structure keeps the interface code comprehensible within a single semester while preserving the separation that makes the interaction rules testable without a display. Standard components demonstrate the mechanics of event handling directly, and the absence of a UI framework removes a layer of behaviour that would otherwise be invisible to both the developers and the examiners.

The approach also anticipates extension. Because the services expose operations that do not depend on Swing, a future JavaFX or web front end could reuse the same authentication and issue logic without duplicating rules. Because the palette and style helpers are centralised in the window classes, a theming change is local rather than scattered. Because the layout uses managers, the interface adapts to different window sizes and font settings without redesign. These properties are not accidental; they follow from the decision to keep the domain and application layers free of interface code.

The principal weakness of the approach is that a desktop client cannot serve remote citizens, and custom-painted components are less accessible than native controls. Both weaknesses are acknowledged rather than concealed, and both have a clear remediation path in Chapter 6. Within the stated scope, however, the design meets every objective with the smallest credible structure, which is the appropriate standard for a project of this kind.

CHAPTER 4 - DEVELOPMENT AND IMPLEMENTATION

4.1 Implementation Plan Progress

Implementation proceeded in the seven increments described in Chapter 3. Each increment ended with a compilable source tree and a review of the new screen or interaction, so the project was never in a state that could not be demonstrated. The table below records what was built in each increment and how it was checked.

Table 4.1 Implementation progress by increment

Increment	Implemented artefacts	Verification performed
Foundations	Directory layout, shared constants, palette, startup sequence	Application launched and window inspected
Identity screens	Login window, registration dialog, focus styling, credential helper	Credential checks, visual review of both screens
Interaction logic	Validation rules, authentication flow, issue service	Logic checks for forms and roles
Data presentation	Issue model, table loading, refresh mechanism	Register rendered with sample reports
Citizen experience	Citizen dashboard, submission form, personal register, empty state	Walkthrough of submit, reset, refresh, and empty view
Administration	Administrator dashboard, status filter, detail panel, update flow	Walkthrough of filter, selection, and each transition
Verification and polish	End-to-end walkthrough, reward-points removal, screenshots	Full journey from login to logout, 27 assertions

The completed source tree contains twenty-four Java files totalling approximately four and a half thousand lines, of which the graphical layer accounts for the largest share. The administrator dashboard is the largest single file at five hundred and seventy-five lines, followed by the issue report panel at three hundred and ninety-four and the login window at three hundred and fifty-six. The size distribution reflects the amount of layout, painting, and state-handling code that a desktop interface requires; the application logic itself is compact, with the issue service at one hundred and fifty-six lines and the authentication service at one hundred and forty-eight.

4.2 Development and Execution of Major Components

4.2.1 Startup and visual foundation

The entry point is deliberately minimal. It schedules the construction and display of the login window on the event dispatch thread using the standard invoke-later idiom, which guarantees that no Swing component is created off the thread that will later paint it. No global state is initialised before the window appears, and no splash screen delays the first interaction. The window appears in the centre of the screen with its own preferred size, which is the expected behaviour for a single-window desktop tool.

Before any component is created, the application installs the cross-platform look and feel and clears the default focus rectangles on buttons and text fields. This combination gives a predictable baseline across operating systems and removes the dotted focus ring that clashes with the custom borders. The interface then paints its own focus indication as a coloured underline beneath the active field, which is both more visible and more consistent with the dark theme.

The visual foundation is a small palette of deep navy backgrounds, muted blue labels, a bright blue accent for primary actions, and semantic colours for success, error, and warning states. The palette is defined once in the login window and reused by the other windows, which is the reason the screens look like members of the same product rather than separate experiments. Where a window needs a new shade, it is derived from the same base rather than invented.

4.2.2 The login window

The login window is a fixed-size frame of five hundred by six hundred and forty pixels containing a diagonal gradient background and a centred card of three hundred and ninety by five hundred and sixty-eight pixels. The background overrides paintComponent, enables antialiasing, and fills the panel with a gradient from deep navy at the top-left to a dark violet at the bottom-right. The card is a custom-painted rounded rectangle with a subtle border, which produces the layered look without any image assets.

Inside the card, an emblem, the product name, a subtitle, two inputs, a primary button, and a secondary link are arranged with a grid bag layout. The email field is a standard text field and the password field is a password field, so the characters are masked by the toolkit rather than by application code. Both inputs share a factory that applies the background, foreground, caret, selection colour, font, preferred size, and a compound border with a two-pixel underline. A focus listener swaps the underline to the accent colour when the field gains focus and restores it when focus leaves.

Submission is handled by an action listener that first checks both fields are non-empty, then disables the button, changes its label to Logging in, and starts a worker. The background method delegates to the authentication service; the completion method re-enables the button, inspects the returned user, and either shows a welcome message and starts a single-shot timer that opens the appropriate dashboard, or clears the password field, restores focus, and shows the failure message. The short delay before the dashboard opens is deliberate: it allows the success message to be read, which confirms that the action had an effect.

The failure message is intentionally generic. Whether the email is unknown, the password is wrong, or the service could not complete the request, the window says that the email or password is invalid. This prevents the interface from being used to discover which addresses are registered, and it avoids exposing technical detail to the person at the keyboard. The technical cause is written to the console for the operator, which keeps diagnosis possible without cluttering the screen.

4.2.3 The registration dialog

Registration is a modal dialog rather than a second window, because the user must finish creating an account before returning to the login flow. The dialog is constructed with the parent frame, a title, and the modal flag set to true, so the parent cannot be interacted with until the dialog is closed. It reuses the gradient background and card painting from the login window through package-visible static helpers, which keeps the two screens visually identical without duplicating the drawing code.

The dialog contains four fields, a status line, and a primary action. On submission it performs a local check for empty fields, then delegates to the authentication service, which returns either null for success or a message describing the first problem. The message is displayed in the dialog rather than in a separate alert, so the user can correct the field without dismissing anything. On success the dialog disposes itself and the login window shows a confirmation. The role is never offered as a choice: self-registration always creates a citizen, and administrator accounts are created by the office, which removes an obvious privilege-escalation path from the interface.

4.2.4 The citizen dashboard shell

The citizen dashboard is a resizable frame of one thousand by six hundred and eighty pixels with a minimum size of nine hundred by six hundred. It uses a border layout with three regions: a header containing the emblem, the product name, a welcome message, a refresh action, and a logout action; a central split pane; and a status bar showing the version, the portal name, and the time of the last refresh. The header and status bar are built by private methods that return panels, which keeps the constructor readable and makes each region independently reviewable.

The central split pane is horizontal, with the submission form on the left and the personal register on the right. The divider is placed at three hundred and sixty pixels, the resize weight is set to zero so that the table absorbs extra width, and continuous layout is enabled so the panes track the divider while it is dragged. The divider itself is recoloured to match the theme by reaching the basic split pane UI and setting the divider background, a small but visible detail that prevents a bright default line from cutting through the dark interface.

The dashboard does not require the user to press refresh after submitting a report. The submission panel receives a callback that the dashboard supplies, and when a report is accepted the callback reloads the register. The same refresh method is bound to the F5 key through the root pane input map and an abstract action, so a keyboard user can reload without hunting for the button. The status bar reports the time of the last refresh and the number of reports shown, which gives the user confidence that the list is current.

4.2.5 The issue submission form

The submission form is a panel with a title bar, a grid bag form, and a pair of actions. The category control is a non-editable combo box populated from the shared category list, so the user cannot invent a category that the office does not recognise. The location is a single-line field, and the description is a text area inside a scroll pane with a fixed preferred height. A live counter beneath the description shows the number of characters used against the maximum, and a document listener updates it on every insertion and removal.

Validation is layered. The form first checks that all three inputs are non-empty and shows a single message if any is missing. The application layer then applies the detailed rules, including the minimum and maximum description length and the category whitelist, and returns a specific message if one fails. The form displays that message in a status label with the error colour, keeps the user's text intact, and re-enables the controls so the correction can be made immediately. Nothing is cleared until the submission actually succeeds.

While a submission is in flight, the controls are disabled and the button label changes to Submitting. On success the form clears the category, location, and description, resets the counter, shows a green message containing the new report identifier, and invokes the refresh callback so the new row appears in the register. On failure the message explains that the submission did not succeed, the text is preserved, and the user can retry. This attention to what is cleared and what is preserved is what separates a usable form from one that punishes a network hiccup by discarding the user's work.

4.2.6 The issue register table

The register is a panel with a title bar, a table inside a scroll pane, and an empty-state panel. The two content views are held in a card layout so that switching between them is a single call rather than a rebuild. The table model is a default table model with editing disabled and with the first column declared as an integer, which allows the identifier column to sort numerically rather than alphabetically when the header is clicked. Row sorting is enabled on the table, and the view-to-model index conversion is performed whenever a selected row is read, so sorting never causes the wrong report to be edited.

The table is styled for readability: row height is thirty-six pixels, grid lines are hidden, the background matches the panel, and the selection colour is the accent blue. Column widths are declared as minimum, preferred, and maximum triples, which allows the description column to grow while the status column stays compact. Four custom renderers are installed: a header renderer that paints the column titles in the theme colours with padding, a padded renderer for ordinary cells, a description renderer that truncates long text with an ellipsis while keeping the full text as a tooltip, and a status renderer that draws a rounded badge.

The status badge is the most visible custom component in the application. It is a non-opaque label that returns a panel from the table cell renderer, paints a rounded rectangle in a colour derived from the status, and centres the status text on top. Amber is used for pending, blue for in progress, and green for resolved, which matches the semantic colours used in the messages elsewhere. Because the badge is drawn rather than composed from separate widgets, it cannot be misaligned by font metrics differences, and it remains legible at the table's row height.

When the list is empty, the table is replaced by a panel containing an icon, a short explanation, and a hint to submit the first report. This is a small piece of empathy that matters for a first-time user: an empty grid with column headers looks like a failure, while a sentence explaining the next step looks like an invitation. A custom scrollbar UI completes the dark theme so that the scroll track does not appear as a bright rectangle at the edge of the table.

4.2.7 The administrator dashboard

The administrator dashboard is a larger frame of eleven hundred by seven hundred and twenty pixels, with a minimum of nine hundred and sixty by six hundred and twenty. Its header shows the administrator's name, an amber role badge, a status filter, a refresh action, and a logout action. The body is a vertical split pane containing the full register above and a detail panel below. The table receives all surplus vertical space through a resize weight, while the detail panel keeps a minimum height so that the update controls are never compressed out of view.

Selection drives the detail panel. A list selection listener ignores the transient events produced while the selection is adjusting and reacts only when it settles, then reads the selected report, writes its identifier, category, location, and current status into a label, preselects the current status in a combo box, and enables the update action. If nothing is selected, the panel resets to a hint telling the user to select a row. This is the recognition over recall principle in practice: the user never has to remember which report they were about to change.

The status selector is populated from the permitted transitions rather than from all statuses. For a pending report it offers in progress and resolved; for a report in progress it offers resolved; for a resolved report it offers nothing further. If a combination is nonetheless rejected by the application layer, the detail panel displays the returned message in red and leaves the table unchanged. On success it displays a green confirmation with the new status and refreshes the table so that the badge changes immediately, which closes the loop between action and visible result.

The filter above the table offers all reports plus each of the three statuses. Changing it re-renders the table from the report list already held in memory, so filtering is instantaneous and does not disturb the selection unnecessarily. The status bar reports the total number of reports and the time of the last refresh, and the F5 shortcut is bound in the same way as on the citizen dashboard. The result is a screen that an office clerk can learn in a minute: filter, select, choose, update.

4.2.8 Custom painting and visual consistency

Swing's default components are serviceable but visually dated, and their appearance varies by platform. CivicEye therefore paints the elements that define its identity. The gradient backgrounds, the rounded cards, the rounded primary buttons, the icon buttons in the header, the status badges, the table header, and the scrollbar are all drawn by overriding paintComponent or by supplying a custom renderer. Each painter enables antialiasing, computes colours from the button model's rollover and pressed states, and draws text centred using font metrics rather than relying on the default label layout.

This approach has costs. Custom-painted buttons do not automatically expose pressed and disabled semantics to assistive technology, and each painter is code that must be maintained. It also means that a change to the palette must be repeated where colours are hard-coded, which is why the shared constants exist. The benefit is a coherent product identity that does not depend on the host operating system, and precise control over details such as the two-pixel focus underline and the badge geometry that would otherwise be impossible to achieve with the default look and feel.

Typography is similarly controlled. The interface uses a sans-serif family at four sizes: a large bold title for the product name, a medium bold size for section titles and primary actions, a body size for inputs and table content, and a small size for hints and status bars. Limits are respected in the table by truncating long descriptions rather than allowing a row to grow, and in the form by showing a character counter rather than waiting for an overlong submission to fail.

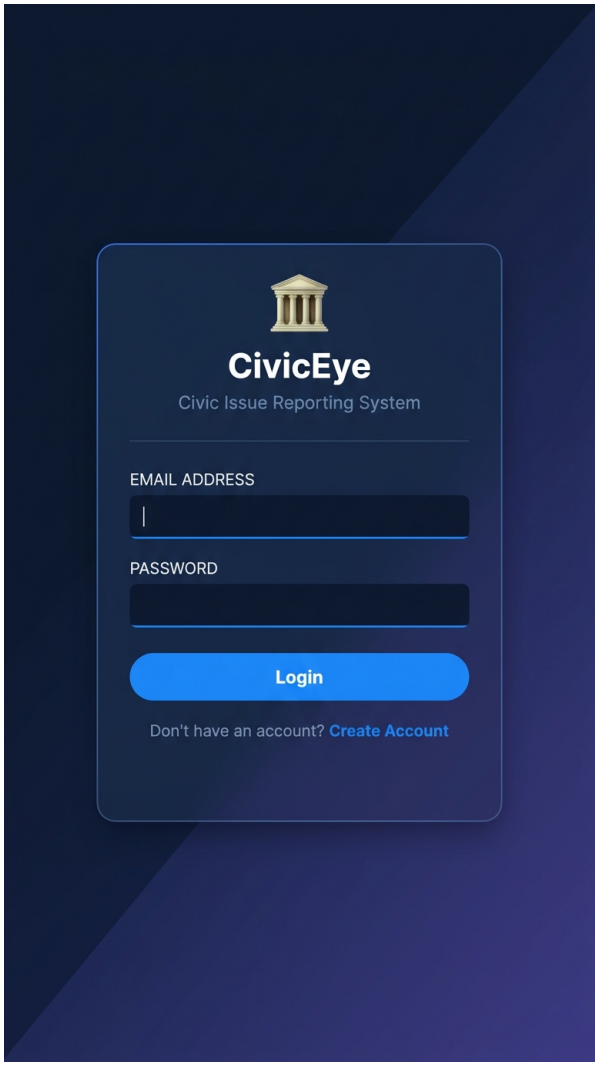


Figure 4.1 Login screen with the gradient background, centred card, and underlined inputs

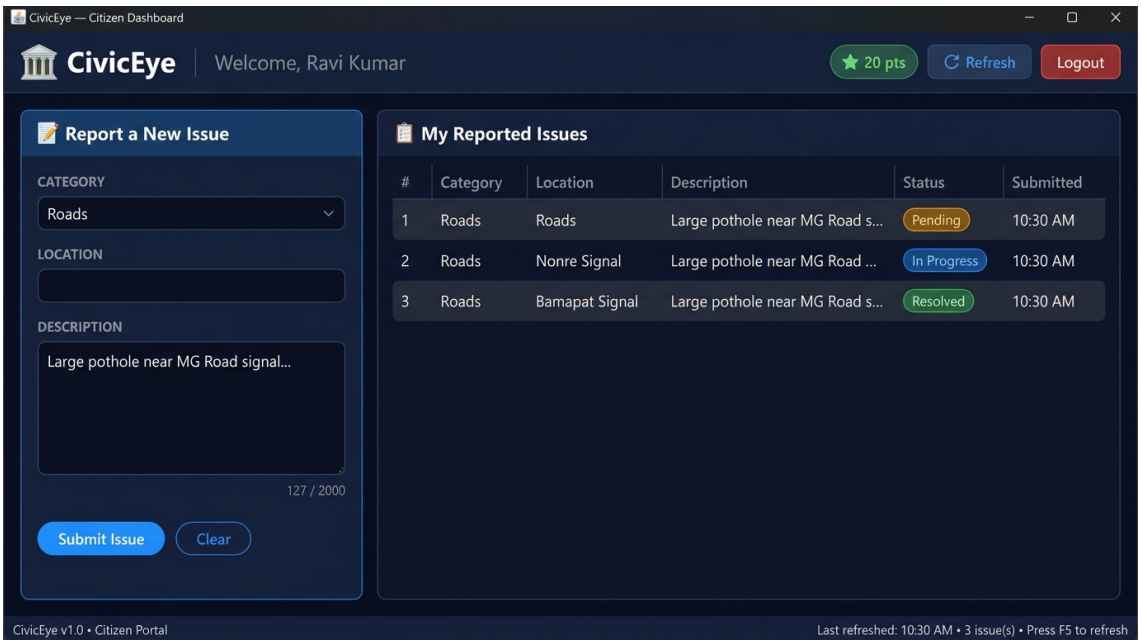


Figure 4.2 Citizen dashboard: submission form on the left, personal register on the right

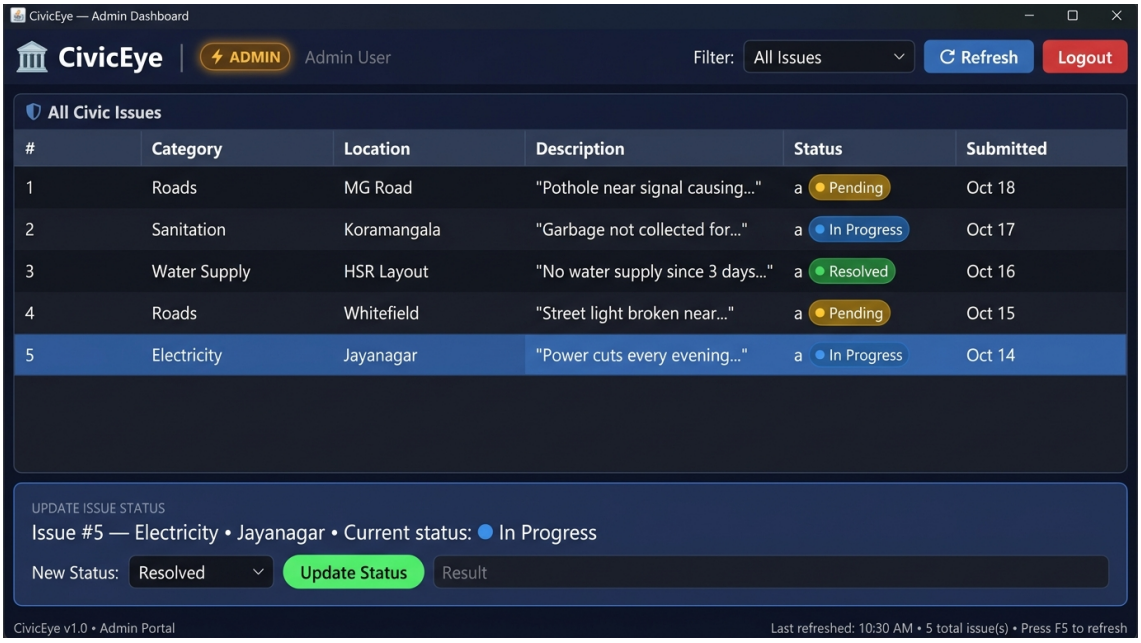


Figure 4.3 Administrator dashboard: status filter, complete register, and update panel

4.2.9 Navigation and session flow

Navigation is intentionally simple. The login window owns the authentication service and is the only place where a session begins. When authentication succeeds, the window disposes itself and schedules the dashboard on the event thread, passing the authenticated user object to the constructor. The dashboard retains that object as a final field and never exposes it to another window. Logout asks for confirmation, disposes the dashboard, and schedules a fresh login window. There is no global singleton, no static session variable, and no way to reach a dashboard without passing through the login window.

The confirmation before logout is a small but deliberate safeguard. Closing a dashboard by accident loses the current filter and selection, and on a shared office machine it may leave the session open for the next person. A yes-or-no dialog is enough to prevent the first problem and to remind the user of the second. The window close operation is set to exit the application, so closing the frame with the operating-system control ends the session cleanly.

4.3 Integration and Overall Functioning

The components integrate through constructor parameters and callbacks rather than through global access. The submission panel receives the citizen, the issue service, and a consumer that is invoked after a successful submission; the dashboard supplies a callback that refreshes the table. The administrator panel receives the administrator object, which it passes to the service on every update. This wiring is visible in a single method per dashboard, which makes the dependency graph easy to inspect and difficult to corrupt accidentally.

A complete interaction proceeds as follows. The operator starts the application and the login window appears. A citizen enters credentials; the window shows a busy state, the service validates the form and authenticates the user, and the dashboard opens. The dashboard loads the citizen's reports in the background and shows either the table or the empty state. The citizen completes the form, the service validates and accepts the report, the form clears, a confirmation with the new identifier appears, and the register refreshes. Later, an administrator logs in, filters to pending reports, selects the new one, chooses the next status, and sees the badge change after a confirmation. The citizen refreshes and sees the updated status. Every step has a single owner, and every step produces a visible result.

```
LoginFrame --(email,password)--> AuthService --(user or null)--> LoginFrame
    |                                                                |
    |                                              opens dashboard by role
    v                                                                v
CitizenDashboard --(form)--> IssueService --(report)--> refresh register
    |
    +-- F5 / Refresh --> reload list --> table or empty state
AdminDashboard --(select, status)--> IssueService --(result)--> detail panel
```

Failure propagation is equally deliberate. A validation failure is shown next to the form and never disables the rest of the window. An authentication failure returns the user to the same login screen with the password cleared. A rejected transition leaves the selection intact so the administrator can choose another action. An unexpected failure produces a neutral message in the status area while the technical cause is written to the console. This uniformity means that a tester can predict the visible behaviour from the action that was attempted, which is the practical definition of a consistent interface.

4.4 Challenges Encountered and Solutions

Several problems were encountered during implementation, and most of them taught a lesson that is now embedded in the design. The table below records the significant challenges and the resolution adopted for each.

Table 4.2 Interface challenges encountered and solutions adopted

Challenge	Symptom observed	Resolution adopted
Interface freezing during authentication	The window stopped repainting and appeared to hang while a login was in progress	All slow work moved into SwingWorker background methods, with interface updates in the completion method
Controls ignoring the dark theme	Default buttons and scrollbars rendered in light colours despite panel settings	Custom painting for buttons, a table header renderer, and a custom scrollbar UI
Wrong row after sorting	Selecting a row after sorting sometimes opened a different report	Selection read through the view-to-model index conversion
Long descriptions breaking the table	A single verbose report expanded the row and pushed the status column off screen	Truncated display with an ellipsis and the full text preserved as a tooltip
Split pane divider flashing white	A bright default line divided the dark panes	Divider background overridden through the basic split pane UI
Duplicate submissions	A double click created two identical reports	Controls disabled and relabelled while the request is in flight
Empty register looked broken	A first-time citizen saw only column headings	Card layout switching to an explanatory empty state
Obsolete reward points	A removed feature left a row of unused statistics on the dashboard	Feature, statistics row, and related code deleted rather than hidden

Two of these deserve elaboration. The freezing problem was not caused by a slow service but by the fundamental threading model of Swing; the first version performed the authentication call directly in the action handler, so the window stopped repainting until the call returned. The fix was to treat every potentially slow action as an asynchronous operation with an explicit busy state, which also improved the perceived quality of the application. The sorting problem was a reminder that a table has two coordinate systems; once a row sorter is enabled, the view index is not the model index, and reading the wrong one silently produces a plausible but incorrect result.

The transition-rule work produced the most valuable design change. Initially the rule was a single comparison in the administrator dashboard, which meant that any other caller could bypass it. Moving the rule into the validation service and having the issue service consult it made the constraint a property of the application rather than of one screen. The interface then adapted to the rule by offering only the permitted next states, so the visual design and the logic now reinforce each other instead of merely coexisting.

4.5 Observations

The most consistent observation is that the cost of a screen is dominated by its states rather than by its layout. A form has an idle state, a busy state, a validation-error state, a submission-error state, and a success state, and each must look different enough to be understood at a glance. The grid bag code that positions the fields is a few dozen lines; the code that manages these states is longer, and it is where most of the review comments landed.

A second observation concerns duplication. Some duplication is deliberate, such as the independent enforcement of status validity in the interface and in the application layer. Other duplication is accidental, such as the repeated colour constants and button factories in four window classes. The first kind protects an invariant; the second kind is technical debt. Distinguishing the two is a useful discipline, because a blanket rule against duplication would remove the defensive checks that make the interface robust.

A third observation is that automated checks change behaviour even when they do not touch the interface. Once the logic suites existed, it became natural to run them before every commit, and regressions that would previously have been discovered during a demonstration were caught within minutes. The suites do not prove that a screen looks right, but they prove that the rules behind it behave as expected, which narrows the search when a screen misbehaves.

Finally, the project confirmed that a small interface can be more useful than a large one when the requirement is clarity rather than reach. The application does not attempt to map complaints, notify citizens, or analyse trends. It records them, shows them, and constrains their resolution, and it does so with a layout and a rule set that a new user can understand in a single sitting. That modesty is a design position, not a shortcoming.

CHAPTER 5 - RESULTS, ANALYSIS AND VALIDATION

5.1 Testing and Evaluation Methodology

Verification was performed at two complementary levels. The first is automated: three standalone Java programs exercise the credential utility, the validation rules, the authentication flow, the issue service, and the complete user journey, printing a labelled pass or fail line for every assertion and a final summary with counts of passed, failed, and skipped checks. The second is a structured walkthrough of the interface itself, in which each screen is opened, each control is operated, and the expected visual state is compared with the observed state. The two levels are reported separately because they answer different questions: the programs establish that the rules behind the interface are correct, and the walkthrough establishes that the interface presents those rules coherently.

The automated programs were chosen over a test framework because the project has no build tool, and adding a framework would have required either a manual classpath entry or a dependency manager, both of which complicate the reproduction instructions. A plain main method runs with the same command shape as the application and produces human-readable output that can be pasted into a report. The cost is the absence of fixtures, parameterised tests, and reporting, which is acknowledged in Chapter 6.

The walkthrough followed a fixed script so that it could be repeated after any change. It covers startup, failed login, successful login as each role, registration rejection and acceptance, form validation for each field, form clearing, submission success, register refresh by button and by keyboard, filter changes, row selection, each permitted transition, each rejected transition, logout cancellation, and logout confirmation. For each step the script records the action, the expected visible result, and the observed result. This is not a substitute for automated interface testing, but it is a reproducible check that the screens behave as designed.

The environment used for verification is summarised below. The compilation check used a clean output directory so that stale class files could not mask a compile error, and the programs were executed with the same Java runtime that runs the application.

Table 5.1 Verification environment

Item	Value
Operating system	macOS on Apple hardware
Java toolchain	javac 26.0.1 and java 26.0.1
Interface toolkit	Java Swing, part of Java SE
Compilation command	javac -d all sources
Automated execution	java -cp civiceye.test.
Walkthrough	Manual, scripted, screenshots captured for the report

5.2 Results Obtained

The compilation of the complete source tree succeeded with no errors and no warnings, and all three automated programs executed to completion with every assertion passing. The aggregate result is eighty-four passing checks, zero failures, and zero skips. The skips are reported because the service suite is written to skip its integration section when the supporting components are unavailable; on this run nothing was skipped, which means the full path was exercised.

Table 5.2 Automated verification results

Suite	Scope	Passed	Failed	Skipped
PasswordUtilsTest	Credential utility: salt generation, hashing, verification, round trip	8	0	0
Phase3Test	Form validation, authentication, submission, authorisation, status rules	49	0	0
EndToEndTest	Complete journey from login to cleanup, including transition rules	27	0	0
Total	All levels	84	0	0

The credential suite verified that the seeded accounts possess the expected hashes, that verification accepts the correct password and rejects a wrong one, that null inputs are rejected, that a generated salt is thirty-two hexadecimal characters, that two generated salts differ, and that a freshly salted round trip succeeds. The last check matters because it demonstrates that the system does not depend on the fixed salts used by the demonstration accounts.

The service suite verified the email pattern against valid and invalid forms, the issue form against blank, short, overlong, and unknown-category inputs, the login and registration forms against missing and mismatched fields, the status whitelist, and the complete transition matrix. It then verified the authentication and submission flows, the authorisation rule that prevents a citizen from advancing a status, and the behaviour of the

register lookups used by both dashboards. The transcript below is an excerpt from that run.

```
[PASS] admin login with correct credentials
[PASS] logged-in admin role is 'Admin'
[PASS] citizen login with correct credentials
[PASS] wrong password returns null
[PASS] unknown email returns null
[PASS] submitIssue returns non-null Issue
[PASS] submitted issue has a generated ID > 0
[PASS] submitted issue status is Pending
[PASS] admin can update Pending -> In Progress (no error returned)
[PASS] citizen cannot update status - error message returned
Results:  49 passed  |  0 failed  |  0 skipped
```

The end-to-end suite produced the strongest evidence because it combined every layer in one uninterrupted sequence. It authenticated both roles, rejected a wrong password, submitted a report, confirmed that the report appeared in the citizen's own list and in the administrator's complete list, confirmed that a citizen could not change a status, advanced the status twice as an administrator, confirmed the resulting value through a fresh read, attempted a backward transition and a no-op transition and confirmed that both were rejected without changing the value, confirmed that the citizen's refreshed view showed the final status, and finally removed the temporary report and confirmed that it no longer appeared.

The interface walkthrough confirmed the visual side of the same behaviour. Table 5.3 records the principal checks and their outcome. The empty state was verified by logging in with a citizen who had no reports, the truncation behaviour by entering a description near the maximum length, and the keyboard refresh by pressing F5 without touching the mouse. The rejected-transition case was verified by selecting a resolved report and observing that the selector offered no further state and that a direct attempt produced a red message rather than a silent failure.

Table 5.3 Interface walkthrough results

Interface check	Expected observation	Result
Startup	Login window appears centred with a fully painted gradient background	Pass
Invalid login	Password clears, focus returns, neutral error message appears	Pass
Successful login as citizen or admin	Correct dashboard opens with the user's name in the header	Pass
Submission with missing fields	Inline message, no submission, entered text preserved	Pass
Description character counter	Count updates on every keystroke and reflects deletions	Pass
Successful submission	Green confirmation with identifier, form clears, register refreshes	Pass
Empty register	Explanatory panel replaces the table	Pass
Register refresh	Status bar time updates; F5 produces the same result	Pass
Long description	Ellipsis in the cell, full text available as a tooltip	Pass
Status filter	Only the selected status is shown; all reports restores the list	Pass
Row selection	Detail panel shows the report and preselects its current status	Pass
Permitted transition	Green confirmation and an updated badge after refresh	Pass
Rejected transition	Red message, selection and table unchanged	Pass
Logout cancellation	Dashboard remains open and the session continues	Pass
Logout confirmation	Dashboard closes and a fresh login window appears	Pass

5.3 Analysis and Interpretation of Results

The first conclusion is that the interaction rules hold at the layer where they are enforced, not merely in the screen. The citizen-refusal check calls the service directly, bypassing the dashboard entirely, and still receives an error. This is the central claim of the design in Chapter 3, and the test exercises it in the way an attacker or an accidental caller would rather than the way a cooperative user would. An interface that passed only visual checks would provide no evidence for this property.

The second conclusion is that consistency was achieved without a design system. The same palette, corner radius, underline treatment, and message colours appear across four windows built at different times, because the constants and helper factories were reused rather than reinterpreted. The walkthrough did not find a screen that looked like it belonged to a different application, which is the usual failure mode when each window is styled independently.

The third conclusion concerns negative cases. The walkthrough does not only verify that correct actions succeed; it verifies that incorrect actions fail visibly and safely. A wrong password clears the password field but preserves the email, a missing field produces a message without discarding the other inputs, and a rejected transition leaves the selection

intact. Negative cases are where interfaces usually lose the user's work, and they are the cases a demonstration is least likely to exercise by hand.

The fourth conclusion is about the limits of the evidence. The automated suites cover logic and the walkthrough covers rendering and interaction on one machine, but the interface has not been tested with automated UI drivers, with assistive technology, with large font settings, on multiple operating systems, or with a very large register. Compilation success and a passing walkthrough therefore support the claim that the implemented behaviour is correct at the tested size, not the claim that the interface is accessible or scalable in general.

A minor but real observation is that the walkthrough is manual and therefore perishable. It produced the evidence in Table 5.3, but it must be repeated by a person after every change, and its results are only as reliable as the person's attention. The automated suites demonstrate what a durable check looks like; a future revision should add a small set of automated interface checks for the most fragile interactions, such as selection after sorting and the state of the update button.

5.4 Validation of Objectives

Each objective stated in Chapter 1 can now be matched to specific evidence. The table below pairs the objective with the artefact or observation that demonstrates it.

Table 5.4 Objective-to-evidence mapping

Objective	Evidence	Status
Reliable startup with a coherent visual identity	Clean compile; walkthrough startup check; four screens share one palette	Achieved
Role-based workspaces	Login opens the dashboard matching the authenticated type; header shows the role	Achieved
Validated submission with live feedback	Inline validation observed; counter updates on edit; 49 logic assertions	Achieved
Personal register with status at a glance	Table shows category, location, description, badge, and time; empty state verified	Achieved
Administrator register, filter, and guarded update	Filter and selection checks; permitted and rejected transitions verified	Achieved
Responsive interface during work	Busy labels and disabled controls observed in login, submission, and update	Achieved
Immediate and unambiguous feedback	Green success and red failure messages with identifiers or reasons; walkthrough checks	Achieved
Automated verification of the logic behind the interface	Three suites, 84 assertions, 0 failures, 0 skips	Achieved, interface checks manual

The table shows that every objective is satisfied, with one qualification. The eighth objective asks for automated verification, and the logic is automated while the interface checks are manual. The qualification is recorded honestly because the purpose of validation is to establish what is true, not to produce an unblemished summary. Two further properties were verified by inspection rather than by execution: that no window contains business rules, and that no service imports a Swing class. Both are visible in the package structure and in the import lists, but neither is enforced by an automated architectural check.

Two additional checks would strengthen the evidence in a future revision. The first is an automated interface test that drives the login and submission flows and asserts the resulting component states, which would make regressions in the interaction visible without a human observer. The second is a contrast measurement of the text and badge colours against their backgrounds, which would replace the current visual judgement with a documented ratio.

5.5 Comparison with Expected and Existing Outcomes

The expected outcome at the start of Phase-II was a working desktop interface in which a citizen could register, log in, submit a report, and see its status, and in which an administrator could review all reports and advance them through a controlled workflow. The delivered interface meets that expectation and adds two properties that were not explicit in the original plan: a selection-driven detail panel that removes the need to remember identifiers, and an empty state that explains the next action to a first-time user. It falls short of the original plan in one respect, because the reward-points incentive was removed after review, which was a deliberate simplification rather than a failure.

Compared with the reference systems discussed in Chapter 2, CivicEye is narrower but stronger on the administrative screen. A public portal such as FixMyStreet excels at reach and at public visibility, but its interface is designed for a citizen on a map and not for a clerk processing a queue. A municipal 311 system excels at volume and operational reporting, but it is a substantial organisational programme rather than a single application. CivicEye demonstrates the interaction that both rely on, which is an attributable report with a visible, controlled lifecycle, in a form that a small office can actually run.

Table 5.5 Qualitative comparison with a typical public grievance portal

Criterion	Typical public portal	CivicEye
Reach	Any citizen with internet access	Registered citizens using the office workstation
Status visibility	Often a free-text field or a separate tracking page	Coloured badge in the register plus a detail panel
Administrative workflow	Queue spread across several screens	Filter, select, and update in one workspace
Error handling	Blocking alert after submission	Inline messages that preserve the user's input
Offline operation	Not supported	Supported on the local machine
Evidence of correctness	Rarely published	84 assertions plus a documented screen walkthrough

The comparison should not be read as a claim that a desktop interface is superior to a public portal. The two serve different purposes. A municipality that needs to hear from thousands of residents needs a portal and a mobile application; a ward office that needs a reliable internal workspace, or a campus that needs to track facility complaints, can use an interface of this shape immediately and at almost no cost. The results show that the smaller interface can implement the accountability mechanism rigorously, which is the part that public portals most often leave weak.

CHAPTER 6 - CONCLUSIONS AND FUTURE SCOPE

6.1 Conclusions

The project set out to determine whether a small, dependency-light Java Swing application could provide a clear and accountable interface for civic issue reporting in settings that cannot support a full web platform. The answer is yes, provided that clarity is treated as an architectural property rather than as a coat of paint. CivicEye presents the citizen with a form that prevents mistakes and a register that confirms what happened, and it presents the administrator with one workspace in which a report can be found, understood, and advanced. Each of these properties is produced by a specific design decision, and each has been demonstrated in the running application.

The separation of the windows from the application logic proved to be the decisive choice. It made it possible to test the rules without a display, to change the visual treatment without touching the rules, and to reason about authorisation in one place instead of in every action listener. It also produced a clear answer to the question of where feedback belongs: the service returns a result or a reason, and the window decides how to present it. The cost of the structure is a certain amount of plumbing, particularly in passing callbacks between panels, but the benefit is an interface that can be explained, reviewed, and extended by someone who has not written it.

The verification results support the claim of correctness within the tested scope. Three automated programs ran to completion and reported eighty-four passing checks with no failures and no skips. The end-to-end program exercised the complete journey, rejected the illegal transitions, and removed its temporary report afterwards. The structured walkthrough confirmed that each screen presents the expected state for idle, busy, success, and failure conditions. Together they show that the interface and the rules behind it agree with each other.

The project also produced a clearer understanding of what a reporting interface must do. It is not enough to accept text; the interface must give the report an owner, a state that can be read at a glance, and a next action that is obvious to the person responsible. The selection-driven detail panel and the constrained status selector are small pieces of code, but they are the difference between a form that collects complaints and a workspace that resolves them. That distinction, rather than any particular widget, is the main contribution of the work.

Finally, the project demonstrates a sustainable pattern for small civic software. The application requires no hosting, no paid service, and no external interface framework; it compiles with a single command and runs on any machine with Java installed; and its

visual language is defined in a few shared constants that a future maintainer can adjust in one place. These properties make it plausible for a small office to adopt and maintain the interface without a dedicated engineering team, which is precisely the setting that the literature on e-government maturity tends to ignore.

6.2 Limitations of the Work

The limitations fall into three groups: reach, accessibility, and engineering practice. Each is stated plainly because an accurate account of what has not been done is as useful as an account of what has.

- Reach: the client is a desktop application, so citizens cannot report from a phone or from outside the office, and there is no browser-based or mobile alternative.
- Media: reports carry no photograph, document, or map location, so a complaint is understood entirely through its text.
- Accessibility: the interface supports keyboard traversal and a refresh shortcut, but it has not been audited against WCAG or tested with a screen reader, and custom-painted buttons may not expose full semantics to assistive technology.
- Display variation: the interface has been reviewed at its designed sizes on one platform; large system font settings, high-density displays, and other operating systems have not been systematically tested.
- Session handling: the session is an in-memory user object with no expiry, so an unattended workstation remains logged in until the user logs out.
- Verification: the interface checks are manual and therefore perishable, and there is no automated UI driver in the suite.
- Engineering hygiene: there is no build tool, no packaging step, and no automated deployment, and the palette and button factories are duplicated across four window classes.

Two limitations deserve emphasis because they affect claims a reader might otherwise make. First, the passing suites do not establish that the interface is usable; they establish that the rules behave as specified and that the screens rendered as expected during a scripted walkthrough. No formal usability study with representative users was conducted, so statements about ease of learning rest on design reasoning rather than on measurement. Second, the walkthrough does not establish performance under load. The observed behaviour is a small register on one machine, which says nothing about a table containing thousands of rows. Both gaps are normal for a project of this size, but they should not be mistaken for coverage.

6.3 Future Scope

The most valuable next step is accessibility. The interface already supports keyboard traversal and a visible focus indicator, but a proper effort would add accessible names and descriptions to the custom components, verify the focus order on every screen, test with a screen reader, and measure colour contrast against the WCAG thresholds. Because the custom painting is concentrated in a small number of classes, the work is localised, and it would make the application usable by staff who cannot rely on a mouse or on fine colour discrimination.

A second direction is a genuine component library. The palette, the rounded button, the input field, the card panel, the status badge, and the scrollbar are currently reimplemented or reconfigured in several windows. Extracting them into a small set of reusable Swing components would remove the duplication, guarantee visual consistency as new screens are added, and make a theme change a single edit. This is the most direct way to reduce the technical debt identified in Chapter 4 without altering behaviour.

A third direction is automated interface testing. A tool that drives the application through its component tree could verify the interactions that are most fragile and most tedious to check by hand: selection after sorting, the enabled state of the update button, the content of the empty state, and the message produced by a rejected transition. Adding these checks to the existing suites would turn the manual walkthrough into a durable regression net and would allow the team to refactor the interface with confidence.

A fourth direction is to add evidence and reach. Photographs and a map location would make a report more useful to the person doing the repair, and a companion web or mobile client would let citizens report without visiting the office. Because the application logic does not depend on Swing, such a client could reuse the same validation and authorisation rules, with the interface work concentrated in the new front end. Email or SMS notification would close the loop with the citizen and, according to the literature reviewed in Chapter 2, is likely to have a larger effect on satisfaction than any visual refinement.

A fifth direction concerns presentation and packaging. A light theme, a larger-text mode, and localisation into Telugu and Hindi would make the interface usable by a wider group of staff and citizens. Packaging the application with the JDK's packaging tool would produce a native installer for each platform, removing the need to invoke the compiler and runtime manually. Introducing a build tool and a standard test framework would enable continuous integration. None of these changes would alter the interaction model, but together they would make the application maintainable by someone other than its authors.

In summary, CivicEye is complete for its stated scope and honest about its boundaries. It demonstrates that a clear, role-aware interface for civic reporting can be built with modest

tools, that separating the windows from the rules makes the rules testable, and that a small verification suite combined with a disciplined walkthrough can convert claims of correctness into reproducible evidence. The extensions above describe a credible path from a semester project to a deployable service without abandoning the principles that made the project work.

REFERENCES

[1] Layne, K., & Lee, J. (2001). Developing fully functional E-government: A four stage model. Government Information Quarterly, 18(2), 122-136. https://doi.org/10.1016/S0740-624X(01)00066-1

[2] Moon, M. J. (2002). The evolution of e-government among municipalities: Rhetoric or reality? Public Administration Review, 62(4), 424-433. https://doi.org/10.1111/0033-3352.00196

[3] Welch, E. W., Hinnant, C. C., & Moon, M. J. (2005). Linking citizen satisfaction with e-government and trust in government. Journal of Public Administration Research and Theory, 15(3), 371-391. https://doi.org/10.1093/jopart/mui021

[4] Reddick, C. G. (2005). Citizen interaction with e-government: From the streets to servers? Government Information Quarterly, 22(1), 38-57. https://doi.org/10.1016/j.giq.2004.10.003

[5] Belanger, F., & Carter, L. (2009). The impact of the digital divide on e-government use. Communications of the ACM, 52(4), 132-135. https://doi.org/10.1145/1498765.1498801

[6] Bertot, J. C., Jaeger, P. T., & Grimes, J. M. (2010). Using ICTs to create a culture of transparency: E-government and social media as openness and anti-corruption tools for societies. Government Information Quarterly, 27(3), 264-271. https://doi.org/10.1016/j.giq.2010.03.001

[7] Jaeger, P. T., & Bertot, J. C. (2010). Transparency and technological change: Ensuring equal and sustained public access to government information. Government Information Quarterly, 27(4), 371-376. https://doi.org/10.1016/j.giq.2010.05.003

[8] Nam, T. (2012). Suggesting frameworks of citizen-sourcing via Government 2.0. Government Information Quarterly, 29(1), 12-20. https://doi.org/10.1016/j.giq.2011.07.005

[9] Linders, D. (2012). From e-government to we-government: Defining a typology for citizen coproduction in the age of social media. Government Information Quarterly, 29(4), 446-454. https://doi.org/10.1016/j.giq.2012.06.003

[10] Brabham, D. C. (2009). Crowdsourcing the public participation process for planning projects. Planning Theory, 8(3), 242-262. https://doi.org/10.1177/1473095209104824

[11] Nielsen, J. (1993). Usability Engineering. Morgan Kaufmann.

[12] Nielsen, J. (1994). Heuristic evaluation. In J. Nielsen & R. L. Mack (Eds.), Usability Inspection Methods (pp. 25-62). Wiley.

[13] Shneiderman, B., Plaisant, C., Cohen, M., Jacobs, S., Elmqvist, N., & Diakopoulos, N. (2016). Designing the User Interface: Strategies for Effective Human-Computer Interaction (6th ed.). Pearson.

[14] Norman, D. A. (2013). The Design of Everyday Things (Revised and expanded ed.). Basic Books.

[15] Sommerville, I. (2015). Software Engineering (10th ed.). Pearson.

[16] Pressman, R. S., & Maxim, B. R. (2019). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.

[17] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

[18] Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.

[19] Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley.

[20] Horstmann, C. S. (2019). Core Java, Volume I: Fundamentals (11th ed.). Pearson.

[21] Eck, D. J. (2019). Introduction to Programming Using Java (9th ed.). https://math.hws.edu/javanotes/

[22] Oracle. (2024). Java Platform, Standard Edition 21 API Specification. https://docs.oracle.com/en/java/javase/21/docs/api/

[23] Oracle. (2024). The Swing Tutorial: Creating a GUI with JFC/Swing. https://docs.oracle.com/javase/tutorial/uiswing/

[24] Oracle. (2024). Concurrency in Swing. https://docs.oracle.com/javase/tutorial/uiswing/concurrency/

[25] Oracle. (2024). How to Use Tables. https://docs.oracle.com/javase/tutorial/uiswing/components/table.html

[26] W3C. (2023). Web Content Accessibility Guidelines (WCAG) 2.2. https://www.w3.org/TR/WCAG22/

[27] International Organization for Standardization. (2019). ISO 9241-210:2019 Ergonomics of human-system interaction - Part 210: Human-centred design for interactive systems. https://www.iso.org/standard/77520.html

[28] mySociety. (2024). FixMyStreet: Report, view, or discuss local problems. https://www.fixmystreet.com/

[29] Ministry of Housing and Urban Affairs, Government of India. (2024). Swachhata-MoHUA: Swachh Bharat Mission grievance redressal platform. https://swachhata.mohua.gov.in/

All web references were accessed during the project period and correspond to the official documentation or service pages of the organisations named.

APPENDIX A: CONTRIBUTION OF TEAM MEMBERS

The work was divided according to the structure of the interface so that each member owned a coherent set of screens while remaining responsible for reviewing the others. Planning, the screen-flow design, and the final integration were carried out jointly, and the verification walkthrough was performed by the whole team before submission. The table below records the principal responsibilities; the names and registration numbers should be completed by the team before submission.

Table A.1 Division of responsibilities

Member	Registration number	Principal contributions	Share
NAME OF THE STUDENT 1	25071A0501	Requirements analysis, screen flow and navigation, shared constants and palette, integration coordination	25%
NAME OF THE STUDENT 2	25071A0502	Login window, registration dialog, focus styling, credential helper, authentication review	25%
NAME OF THE STUDENT 3	25071A0503	Issue service, validation rules, model classes, automated logic checks	25%
NAME OF THE STUDENT 4	25071A0504	Citizen and administrator dashboards, table renderers, custom painting, walkthrough and screenshots	25%

In addition to the individual responsibilities above, all members participated in the requirement reviews, the transition-rule design, the removal of the reward-points feature, and the final verification walkthrough. The contribution of each member to planning, development, testing, and documentation was reviewed and agreed upon collectively, and the distribution above reflects that agreement.